

Carreras críticas en el problema del productor-consumidor

Nuria Guerra Casal Adrian Gijon Yubero
Grao en Enxeñaría Informática

1. Introducción

El problema del productor-consumidor es un problema clásico de sincronización: dos procesos comparten un buffer de tamaño fijo N , donde el productor deposita datos y el consumidor los extrae. Sin ningún mecanismo de sincronización, ambos procesos pueden acceder simultáneamente a las variables compartidas, produciendo **carreras críticas**: situaciones en las que el resultado del programa depende del orden impredecible en que el sistema operativo planifica la ejecución de cada proceso.

El objetivo de este apartado es precisamente demostrar que estas carreras críticas existen y son observables. Para ello se han implementado dos programas, `prod.c` y `cons.c`, que operan sobre un buffer compartido en memoria sin ninguna sincronización, y se han introducido llamadas a `sleep()` en los puntos críticos para ampliar la ventana de tiempo en la que puede ocurrir la carrera, aumentando así la probabilidad de que se manifieste en cada ejecución.

2. Estructura

Los dos programas comparten una estructura de datos mapeada en memoria mediante `mmap()` sobre el fichero `buffer.dat`. Esta técnica permite que dos procesos independientes (no siendo hilos) accedan a la misma región de memoria física. La estructura compartida es la siguiente:

```
1 typedef struct {  
2     char pila[TAMANO_BUFFER]; // buffer LIFO de caracteres (N=10)  
3     int cima; // indice del siguiente hueco libre  
4     int rematou; // indicador de que el productor termino de leer  
5 } DatosCompartidos;
```

Listing 1: Estructura compartida entre productor y consumidor

El campo `cima` es el que provoca las carreras críticas: tanto el productor como el consumidor lo leen y modifican sin ningún control, lo que puede llevar a estados inconsistentes. El campo `rematou` permite al consumidor saber cuando el productor ha terminado de leer el fichero y el buffer ya no recibirá más datos (posteriormente se eliminará), momento en el que el consumidor puede finalizar.

El productor crea el fichero de memoria compartida con `open(..., O_CREAT)` y lo redimensiona con `ftruncate()`, mientras que el consumidor espera activamente con `stat()` hasta que el fichero existe para abrirlo (esto puede ser prescindible pero se deberá estar pendiente de que el productor haya creado el archivo). Esta coordinación en el arranque es necesaria porque los dos programas se ejecutan en terminales distintas y no hay ninguna sincronización de inicio.

3. Funcionamiento

3.1. Productor (prod.c)

El productor lee el fichero carácter a carácter con `fgetc()`, llama a `produce_item()` para contabilizar las vocales leídas, e inserta cada carácter en el buffer mediante `insert_item()`. La función de inserción realiza primero una espera activa si el buffer está lleno, y después ejecuta la siguiente secuencia:

```
1 //lee cima y guarda la posicion antes del sleep
2 int posicion = datos->cima;
3 printf("cima=%d  productor intenta insertar '%c' en pila[%d]\n", ...);
4
5 sleep(2); // <-- ventana de carrera: el consumidor puede modificar cima aqui
6
7 //usa la posicion guardada, ignorando cambios de cima
8 datos->pila[posicion] = caracter; // puede ser una posicion obsoleta
9 datos->cima = posicion + 1; // sobrescribe lo que hizo el consumidor
```

Listing 2: Región crítica en `insert_item()` – carrera crítica intencionada

El `sleep(2)` entre los dos pasos es el mecanismo que amplifica la carrera. Durante esos 2 segundos el consumidor puede modificar `cima`, pero cuando el productor despierte usará el valor antiguo que guardo en `posicion`, escribiendo en una posición que ya no corresponde a la cima real.

3.2. Consumidor (cons.c)

El consumidor funciona de forma simétrica. Espera activamente si el buffer está vacío y sale del bucle cuando `rematou==1` con el buffer vacío. La función `remove_item()` aplica el mismo patrón de dos pasos con un `sleep()` en medio:

```
1 //calcula la posicion antes del sleep
2 int posicion = datos->cima - 1;
3
4 sleep(2); // <-- ventana de carrera: el productor puede modificar cima aqui
5
6 //usa la posicion guardada, ignorando lo que hizo el productor
7 char caracter = datos->pila[posicion];
8 datos->pila[posicion] = '-';
9 datos->cima = posicion;
```

Listing 3: Región crítica en `remove_item()` – carrera crítica intencionada

Al finalizar, el productor pone `rematou=1` y espera activamente hasta que `cima==0` antes de liberar la memoria, garantizando que el consumidor haya terminado antes de que se desmapee la región compartida.

4. Ejecución

```
# Compilar ambos programas
gcc -o prod prod.c
gcc -o cons cons.c

# Terminal 1: lanzar el productor primero (crea el buffer compartido)
./prod texto.txt

# Terminal 2: lanzar el consumidor después
./cons
```

Es importante lanzar el productor antes que el consumidor, ya que es el productor quien crea e inicializa el fichero de memoria compartida. El consumidor espera automáticamente hasta que el

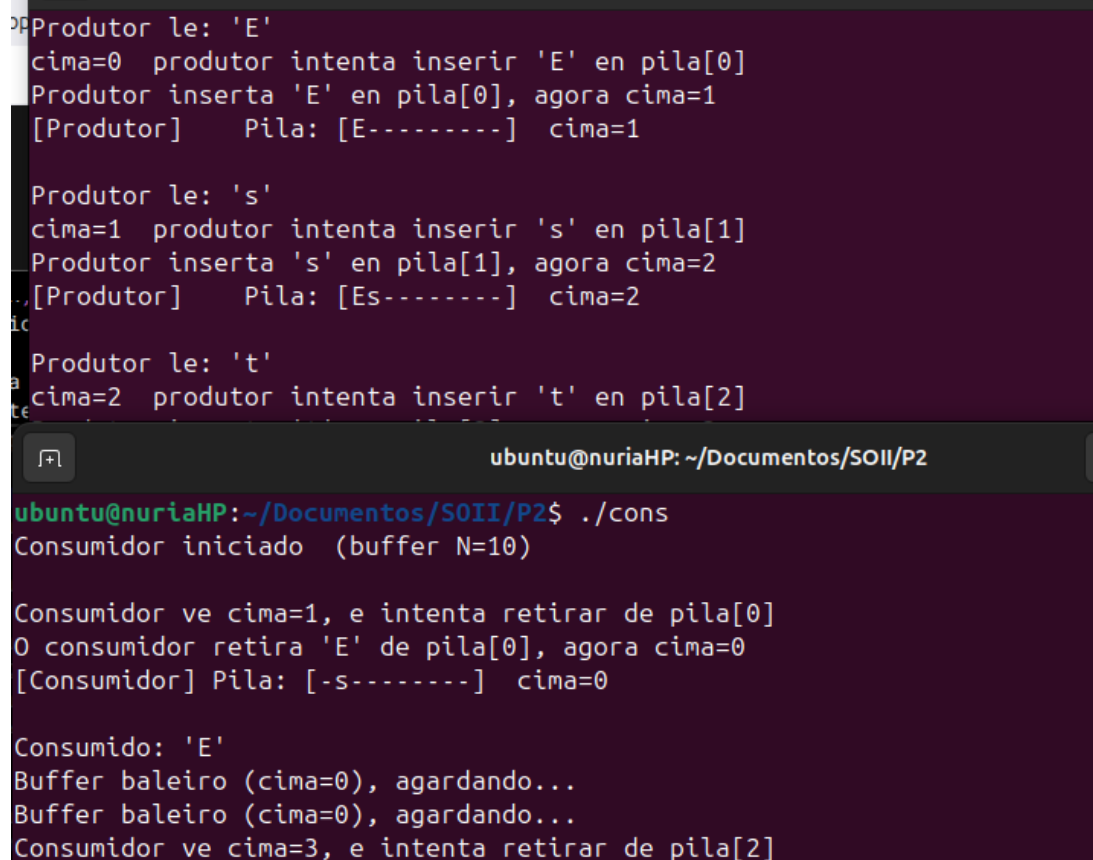
fichero existe. Ambos programas terminan solos al procesar todos los caracteres del fichero.

5. Observacion de las carreras criticas

La carrera critica se produce cuando productor y consumidor ejecutan la parte justo anterior a los sleeps casi simultaneamente y luego la posterior se solapa. Por ejemplo:

1. El productor lee `cima=1` y guarda `posicion=1`. Duerme 2s.
2. El consumidor lee `cima=1` y guarda `posicion=0`. Duerme 2s.
3. El consumidor despierta: retira `pila[0]`, pone `cima=0`.
4. El productor despierta: escribe en `pila[1]` y pone `cima=2`.

El resultado es que `cima` salta de 0 a 2 sin haber pasado por 1, dejando `pila[1]` con un dato pero `pila[0]` vacío. Además, el consumidor puede llegar a retirar guiones '-' en lugar de caracteres reales, porque la carrera deja huecos vacíos entre datos válidos.



```
PPProdutor le: 'E'
cima=0 produtor intenta inserir 'E' en pila[0]
Produtor inserta 'E' en pila[0], agora cima=1
[Produtor] Pila: [E-----] cima=1

Produtor le: 's'
cima=1 produtor intenta inserir 's' en pila[1]
Produtor inserta 's' en pila[1], agora cima=2
[Produtor] Pila: [Es-----] cima=2

Produtor le: 't'
cima=2 produtor intenta inserir 't' en pila[2]

ubuntu@nuriaHP: ~/Documentos/SOII/P2
ubuntu@nuriaHP:~/Documentos/SOII/P2$ ./cons
Consumidor iniciado (buffer N=10)

Consumidor ve cima=1, e intenta retirar de pila[0]
O consumidor retira 'E' de pila[0], agora cima=0
[Consumidor] Pila: [-s-----] cima=0

Consumido: 'E'
Buffer baleiro (cima=0), aguardando...
Buffer baleiro (cima=0), aguardando...
Consumidor ve cima=3, e intenta retirar de pila[2]
```

Figura 1: Captura de parte de ejecución del programa en terminal.

La prueba definitiva de que hay carreras críticas es que **el conteo de vocales del productor y del consumidor no coincide** al final: el productor ha leído más vocales de las que el consumidor ha podido consumir correctamente, porque algunos datos se han perdido o sobrescrito durante las carreras.

6. Conclusiones

Este apartado demuestra de forma práctica que el acceso concurrente a variables compartidas sin sincronización produce resultados erróneos e impredecibles. La variable `cima`, que actúa como índice de la pila LIFO, es el recurso compartido crítico: cualquier acceso a ella puede dejar el

```
4Produtor, vogais contadas:
    a=0   e=5   i=1   o=5   u=3
ubuntu@nuriaHP:~/Documentos/SOII/P2$

[Consumidor] Pila: [----.--dor]  cima=0

Consumido: '-'

Consumidor, vogais contadas:
    a=0   e=5   i=1   o=2   u=2
```

Figura 2: Captura de conteo de vocales.

buffer en un estado inconsistente. Se puede observar como el consumidor retira guiones en lugar de datos reales, el valor de `cima` realiza saltos anómalos, y los conteos finales de vocales divergen entre productor y consumidor.